Idea Overview

1. Tideline (reshaped for software-only)

One-liner. Ingests NOAA buoy feeds, Scripps oceanographic data, and satellite SST to predict marine heatwaves 5-7 days ahead at reef and fishery granularity, with a forecasting dashboard and API for conservation orgs and aquaculture operators.

Prize stack: Scripps Data, AWS or GCP, Databricks, Impulse, NVIDIA Brev.dev (model training), Marimo/Sphinx (notebooks + docs), IEEE Info Theory (multi-source signal fusion), General Challenge, Entrepreneurship. That's 8-9 tracks in play.

Architecture. Kinesis or Pub/Sub ingests NOAA NDBC buoy feeds, CalCOFI oceanographic data, and MODIS SST rasters. Feature engineering in Databricks with Delta Lake for versioned snapshots. Forecasting model (gradient-boosted ensemble plus a temporal transformer for the SST raster features) trained on Brev.dev GPUs, tracked via Impulse. Inference API on AWS Lambda or Cloud Run. React dashboard with a map showing predicted heatwave probability by region, confidence intervals, and historical backtest overlay. Marimo notebook as the reproducible analysis artifact, Sphinx site as the documentation.

Moat. Fusing heterogeneous ocean data sources with mismatched spatial and temporal resolutions is real signal-processing work. Most teams will use one data source and a basic model. Scripps judges will recognize whether the data fusion is genuine or cosmetic.

24-hour plan. Hours 0-4: data pipeline for 2023 Pacific Blob heatwave window, all sources landing in Databricks. Hours 4-12: feature engineering, baseline model, temporal transformer. Hours 12-20: inference API, dashboard, backtest visualization. Hours 20-24: Marimo notebook, Sphinx docs, demo script. Cut aggressively: real-time ingestion (use pre-downloaded rasters), multi-region (pick California coast), fancy auth.

Why it wins. Scripps prize is described as "thoughtful, impactful, effective use of data" and this idea is a love letter to that description. The climate narrative carries the entrepreneurship angle (NOAA estimates marine heatwaves cost global fisheries $2B+ annually). Multi-source fusion gives IEEE Info Theory a real hook. Databricks + Impulse + Brev.dev all do genuine work, not decoration.

Orthogonal fit: None. Accept the trade.

Specifics

Tideline — Full Project Specification

The product in one paragraph

Tideline is a marine heatwave forecasting system that fuses NOAA buoy sensor data, Scripps oceanographic observations, and satellite sea surface temperature rasters to predict marine heatwaves 5-7 days ahead at coastal-region granularity. Users (fisheries managers, aquaculture operators, reef conservation orgs) get a web dashboard with a probability map, confidence intervals, and a historical backtest proving the model works. The system ships with a reproducible Marimo notebook, Sphinx documentation, and an inference API so the analysis is auditable end-to-end.

The thing that makes this a winning hackathon project, not just a good one: the three data sources have genuinely different spatial resolutions, temporal cadences, and noise characteristics. Fusing them properly is real signal processing work, not pipeline glue. That's the technical story. The climate and fisheries angle is the business story. Both hold up to cross-examination.

What a marine heatwave actually is (context you'll need for judges)

A marine heatwave (MHW) is defined by Hobday et al. 2016 as a period where sea surface temperature exceeds the 90th percentile of the local climatology for at least 5 consecutive days. The climatology is computed from an 11-day window centered on each calendar day, using a 30-year historical baseline (typically 1983-2012).

Categories: Moderate (1x-2x the 90th percentile anomaly), Strong (2x-3x), Severe (3x-4x), Extreme (>4x).

Why this matters for your pitch: marine heatwaves caused the Pacific Blob (2013-2016) which killed millions of seabirds, collapsed salmon fisheries, and triggered the largest toxic algal bloom in West Coast history. NOAA estimates the direct economic cost at $3-4B. The 2023 North Atlantic MHW was unprecedented. These events are increasing in frequency and severity. Prediction lead time of 5-7 days gives fisheries and aquaculture operators time to move equipment, harvest early, or deploy mitigations.

Data sources

NOAA NDBC buoys. Real-time and historical sensor data from moored buoys along the US coast. Each buoy reports sea surface temperature, air temperature, wind, wave height at hourly cadence. Access via `https://www.ndbc.noaa.gov/` and the station data endpoints. For a hackathon, download historical data for ~20 buoys along the California coast covering 2015-2023.

Scripps CalCOFI. California Cooperative Oceanic Fisheries Investigations — quarterly research cruise data with temperature/salinity/chlorophyll profiles at 75 stations off the California coast. Lower cadence (quarterly) but deeper ocean coverage. Access via `https://calcofi.org/data/`. This is your Scripps-prize hook.

NOAA Coral Reef Watch or NASA MODIS SST. Satellite sea surface temperature rasters at 5km resolution, daily. Coral Reef Watch specifically publishes MHW-relevant products including DHW (Degree Heating Weeks) already computed. Access via `https://coralreefwatch.noaa.gov/product/` or NASA Earthdata. For the hackathon, pre-download a specific region and date range so you're not fighting a satellite API at hour 20.

Target prediction region. California coast, specifically the Southern California Bight (roughly 32°N-35°N, 117°W-121°W). Reasons: high buoy density, CalCOFI coverage, clear historical MHW events in your backtest window (2014-2016 Blob, 2019 warm anomaly), and it's Scripps' backyard which judges will appreciate.

System architecture

The architecture has four layers: ingestion, feature engineering, modeling, and serving. Each layer maps to a specific cloud service that's also a prize track, which is how you get the stacking to feel natural.

Ingestion. A Python service pulls historical data from the three sources and lands it in cloud storage (S3 or GCS). For a hackathon, you run this once during setup, not continuously. The pipeline produces three raw datasets: `buoys.parquet`, `calcofi.parquet`, `sst_rasters/` (a directory of NetCDF files).

Feature engineering in Databricks. This is where the multi-source fusion happens, and it's the technically interesting part. You build a unified daily feature table where each row is (date, lat, lon) and columns are:

- Buoy-derived features: nearest-buoy SST, SST anomaly vs climatology, 7-day and 30-day rolling means, inverse-distance-weighted SST from the 3 nearest buoys
- CalCOFI-derived features: quarterly-interpolated subsurface temperature at 50m and 100m depth, thermocline depth, chlorophyll concentration
- Satellite-derived features: gridded SST at the target cell, SST gradient magnitude (spatial derivative), DHW accumulation, persistence (days since last cold anomaly)
- Climatology features: day-of-year, 30-year mean SST for this cell and day
- Lag features: the target variable lagged 1, 3, 7 days (since MHWs are autocorrelated)

The fusion trick is that these sources have different native resolutions. Buoys are point observations at hourly cadence. CalCOFI is quarterly profiles at station locations. Satellite rasters are daily gridded. You handle this by defining a target grid (say, 10km cells covering the Bight), then for each cell and each day, computing features by the appropriate method: inverse-distance weighting for buoys, temporal interpolation for CalCOFI, nearest-pixel for satellite. This is the part IEEE Info Theory judges will appreciate if you explain it clearly — you're reconciling observations with different support and noise characteristics into a unified feature vector.

Use Delta Lake tables in Databricks for versioning so your backtest is reproducible.

Modeling. Two models that you ensemble.

The first is a gradient-boosted model (LightGBM or XGBoost) on the tabular features above. Target: binary classifier predicting "will this cell be in MHW status in N days" for N in {1, 3, 5, 7}. You train four separate models, one per lead time. This is your baseline and it will probably get you 70-80% of the performance.

The second is a small temporal convolutional network or transformer on the satellite SST raster sequence. Input: a (T=30, H, W) tensor of SST values for the preceding 30 days. Output: per-cell MHW probability at the target lead time. This captures spatial propagation patterns that the tabular model misses. Train on NVIDIA

Brev.dev GPUs — this is your Brev.dev prize hook and it's real work because raster training is genuinely GPU-bound.

Ensemble: simple weighted average of the two model probabilities. Log the ensemble weights as a hyperparameter you tune on a validation split.

Track training runs on Impulse AI platform. Impulse is described as having an MLE agent and "best models trained on Impulse" — so run your hyperparameter sweep through their platform and cite it.

Evaluation: backtest on held-out 2023 data. Metrics: AUC-ROC, precision at 90% recall, Brier score, and a lead-time-specific accuracy chart. Include a confusion matrix for MHW-vs-not at each lead time.

Serving. FastAPI backend on AWS Lambda (via API Gateway) or GCP Cloud Run. Two endpoints: `/forecast?lat=&lon=&date=` returns JSON probabilities and confidence intervals for each lead time. `/backtest?region=&start=&end=` returns historical predictions vs ground truth. React frontend with Mapbox or Leaflet for the probability heatmap, a timeline slider, and a backtest comparison view.

Documentation. Marimo notebook walks through the full analysis: data loading, feature engineering, model training, backtest evaluation, and a worked example of the 2015 Pacific Blob prediction. Sphinx documentation site covers the API, the model cards, and the data sources with citations. This is your Marimo/Sphinx prize hook.

Prize stack mapping

Scripps Data ($1500, 1st place DJI Neo Drone). CalCOFI is Scripps' flagship dataset. Use it substantively — don't just load it and ignore it. The subsurface temperature features from CalCOFI should measurably improve model performance vs a satellite-only baseline. Include an ablation study in the Marimo notebook showing "model with CalCOFI features" beats "without" by some AUC delta. That ablation is the artifact that wins this prize.

Marimo/Sphinx ($200, 1st place). The notebook is the analytical narrative. Write it like a research paper: intro → data → methods → results → discussion. Sphinx site hosts the API docs and links to the notebook.

Databricks ($1000, JBL Speaker + AirTag + Swag). Use Databricks as the actual feature engineering environment. Delta tables, notebooks, and if you can swing it, an MLflow experiment logged in the Databricks workspace. Don't fake this — judges will check.

AWS Cloud Platform ($1000, Record Player). Lambda for inference, S3 for raw data and model artifacts, API Gateway for the public endpoint, CloudFront for the frontend. Possibly SageMaker for model hosting if Lambda cold-starts hurt.

OR Google Build With AI ($1000, Duffle Bag). Cloud Run for inference, GCS for storage, and integrate Gemini for the dashboard's natural-language summaries ("This week's forecast shows elevated MHW probability off San Diego driven by persistent warm anomalies at 50m depth per CalCOFI observations"). Pick one cloud or the other — you can't realistically compete for both in 24 hours.

My recommendation: go GCP for the Gemini hook, because the NL summary is a great demo moment and Gemini is free to integrate.

NVIDIA Brev.dev ($500, Sennheiser Headphones). Train the raster CNN/transformer on Brev.dev. Document the GPU usage and training runs.

Impulse AI ($800, annual plan + swag). Hyperparameter sweep for the LightGBM models on Impulse. Their prize description emphasizes "best models trained on Impulse" so give them something real to evaluate.

IEEE Information Theory ($500, $125 Amazon gift card). In your pitch, explicitly frame the multi-source fusion as an information-theoretic problem: three observations of the same underlying ocean state, each with different noise and coverage, combined to maximize the mutual information with future MHW status. Reference the Fisher information of each source. This is a 2-minute segment in your pitch that will blow past 99% of submissions.

General Challenge ($1500, DJI Neo Drone). This is the overall "best use of data" prize and Tideline is a textbook fit.

Entrepreneurship tracks. Most Innovative Idea ($200) and Most Viral Idea ($300). Lean into the climate + fisheries narrative for Innovative. For Viral, the demo needs a shareable moment — the 2015 Pacific Blob replay where your model predicts it 5 days in advance is the tweetable clip.

Conservative prize stack estimate: 6-8 prizes. Realistic upside: $4-6K in prize value plus hardware.

24-hour build plan

I'll assume 3 people: you (call it Engineer A, backend/ML), Tanmay or Ashwin as Engineer B (frontend), and the third as Engineer C (data pipeline + docs).

Hour 0-2: setup and data download.

- Everyone: cloud accounts provisioned (GCP + Databricks + Brev.dev).
- Engineer C: start downloading NOAA NDBC data for 20 California buoys, 2015-2023. Start pulling CalCOFI. Start downloading MODIS SST rasters for the Bight, same window. These downloads take hours — parallelize them and keep them running while you do other work.
- Engineer A: scaffold the repo structure, write the feature engineering module skeleton.
- Engineer B: scaffold the React frontend with Mapbox placeholder.

Hour 2-6: feature engineering.

- Engineer C: land all three raw datasets in cloud storage, document the schemas.
- Engineer A: write the feature engineering pipeline in Databricks. Build the unified daily feature table for the full 2015-2023 window. This takes time because you're materializing a large table — aim to have it done by hour 6.
- Engineer B: build the map view with a mock probability overlay so you can iterate on the UI independently.

Hour 6-12: modeling.

- Engineer A: train the LightGBM baselines for all 4 lead times. Get a backtest running on 2023. You should have AUC numbers by hour 10.
- Engineer C: start the Marimo notebook — data loading section, initial EDA plots.
- Engineer B: build the timeline slider and the backtest comparison view. Hook up to mock API.

Hour 12-18: raster model and ensemble.

- Engineer A: spin up Brev.dev GPU, train the temporal CNN on satellite rasters. This is the risky hour — if it doesn't train well, fall back to LightGBM-only and own that decision.
- Engineer C: build the inference API (FastAPI) and deploy to Cloud Run. Wire the frontend to real endpoints.
- Engineer B: polish the UI, add the Gemini NL summary integration, work on the Pacific Blob historical playback view.

Hour 18-22: ensemble, Impulse sweep, docs.

- Engineer A: ensemble the two models, run the ablation study (with/without CalCOFI) for the Scripps prize. Kick off the Impulse hyperparameter sweep.
- Engineer C: finish the Marimo notebook with the worked Blob example. Build the Sphinx site.
- Engineer B: final UI polish, demo script rehearsal.

Hour 22-24: demo prep.

- Everyone: rehearse the 5-minute pitch 3x. Record a backup demo video in case live fails.
- Final commit, submission materials, Devpost writeup.

Aggressive cuts if you're behind schedule:

- Drop the raster model entirely, ship LightGBM only. Saves 4-6 hours, loses maybe 5% AUC, still wins most prizes.
- Drop multi-region, show only one cell prediction with a time series chart instead of a map. Dramatically simpler, still tells the story.
- Drop the Impulse sweep if Impulse onboarding is slow. Lose one prize, keep everything else.

Do not cut: the CalCOFI ablation study, the Marimo notebook, the backtest visualization. These are the artifacts that win prizes.

Demo script (5 minutes)

0:00-0:30. "Marine heatwaves killed $4B of Pacific fisheries in 2015 and nobody saw them coming more than a day in advance. Tideline predicts them 7 days out by fusing buoy, Scripps CalCOFI, and satellite data."

0:30-1:30. Live demo: open the dashboard, click on a San Diego coastal cell for today, show 7-day forecast with confidence intervals. Show the Gemini-generated NL summary.

1:30-2:30. Pacific Blob replay: rewind to July 2015, show the model predicting the onset 5 days before it happened, with the ground truth overlay proving the call was right.

2:30-3:30. Architecture slide: the three data sources, the fusion layer, the ensemble. One-sentence on the info-theoretic framing.

3:30-4:30. Business angle: fisheries managers, aquaculture operators, reef orgs. NOAA's MHW program doesn't do regional forecasting. We do. TAM is $X.

4:30-5:00. Thank the sponsors by name, point at the Marimo notebook and Sphinx docs for reproducibility, close.

Prompts

Structured prompts for Claude Code
These are written as sequential prompts — feed them one at a time, review the output, commit, then move to the next. Each assumes the previous has been completed.

Prompt 1: Repo scaffolding
I'm building Tideline, a marine heatwave forecasting system for a hackathon.
Scaffold a Python monorepo with the following structure:

```
tideline/
├── pyproject.toml          # uv-based project
├── README.md
├── ingestion/              # data download scripts
│   ├── noaa_buoys.py
│   ├── calcofi.py
│   └── satellite_sst.py
├── features/               # feature engineering
│   ├── unified_grid.py     # defines 10km target grid for SoCal Bight
│   ├── buoy_features.py    # inverse-distance weighting from buoys
│   ├── calcofi_features.py # temporal interpolation of quarterly profiles
│   ├── satellite_features.py
│   └── build_feature_table.py  # orchestrator
├── models/
│   ├── climatology.py      # 1983-2012 baseline per cell per day-of-year
│   ├── mhw_labels.py       # Hobday 2016 MHW detection
│   ├── lightgbm_model.py
│   ├── raster_cnn.py
│   └── ensemble.py
├── backtest/
│   └── evaluate.py
├── api/
│   └── main.py             # FastAPI
├── notebooks/
│   └── analysis.py         # Marimo
├── docs/                   # Sphinx
└── frontend/               # separate, React — leave stub
```

Set up pyproject.toml with these deps: pandas, numpy, xarray, netCDF4, scikit-learn, lightgbm, torch, fastapi, uvicorn, pydantic, marimo, databricks-sdk, boto3 OR google-cloud-storage (I'll pick one later — scaffold for GCS), requests, tqdm.

Create a .env.example with placeholders for cloud credentials.
Create a Makefile with targets: setup, ingest, features, train, backtest, serve, notebook, docs.

Write a README that describes the system architecture in 3 paragraphs.
Prompt 2: Ingestion scripts
Implement the three ingestion scripts. Requirements:

ingestion/noaa_buoys.py:

- Download historical standard meteorological data for these NDBC stations:
  46025, 46069, 46086, 46218, 46219, 46221, 46222, 46223, 46224, 46225,
  46231, 46232, 46242, 46253, 46254, 46256, 46258, 46259, 46266, 46268
  (Southern California Bight buoys)
- Pull data from 2014-01-01 to 2023-12-31
- URL pattern:
https://www.ndbc.noaa.gov/view_text_file.php?filename={station}h{year}.txt.gz&dir=data/historical/stdmet/
- Parse the space-separated format, extract columns: YYYY MM DD hh mm WTMP (water temp)
- Resample to daily mean
- Output: parquet file with columns [station_id, lat, lon, date, sst]
- Handle missing data (99.0 sentinel) and stations with no data for some years
- Station lat/lon lookup should be in a constants dict at the top of the file

ingestion/calcofi.py:
- Download CalCOFI bottle data from https://calcofi.org/data/oceanographic-data/bottle-database/
- The CSV is at
https://calcofi.org/downloads/database/CalCOFI_Database_194903-202001_csv_16October2020.zip
- Extract and parse. Columns of interest: Date, St_Line, St_Station, Cst_Cnt,
  Depthm, T_degC, Salnty, ChlorA
- Filter to stations in the SoCal Bight region (lat 32-35, lon -121 to -117)
- Filter to dates >= 2014
- Output: parquet file with columns [date, station, lat, lon, depth_m, temp_c,
  salinity, chlorophyll_a]

ingestion/satellite_sst.py:
- Use NOAA Coral Reef Watch 5km daily SST product
- THREDDS server: https://coastwatch.pfeg.noaa.gov/erddap/griddap/NOAA_DHW.nc
- Query via ERDDAP for variables: CRW_SST, CRW_SSTANOMALY, CRW_DHW
- Bounding box: lat 32-36, lon -122 to -117
- Time range: 2014-01-01 to 2023-12-31
- Save as NetCDF files, one per year (to keep file sizes manageable)
- Output: sst_rasters/{year}.nc

All scripts should:
- Upload final output to GCS bucket (use env var GCS_BUCKET)
- Log progress with tqdm
- Be idempotent (skip if output already exists in GCS)
- Have a main() guarded by if __name__ == "__main__"

Write tests in tests/test_ingestion.py that check the output schemas on a
small sample (1 buoy, 1 year).
Prompt 3: Feature engineering
Implement the feature engineering layer. This is the most important module
technically — it fuses three data sources with different spatial/temporal
resolutions into a unified daily feature table.

features/unified_grid.py:
- Define a 10km target grid covering the SoCal Bight
- Bounding box: lat 32.0-35.0, lon -121.0 to -117.0
- Output: a DataFrame with columns [cell_id, lat, lon] for ~1200 cells

- Function get_grid() returns this DataFrame

features/buoy_features.py:
- For each (cell, date), compute inverse-distance-weighted SST from the 3 nearest buoys that have data on that date
- Also compute: 7-day rolling mean, 30-day rolling mean, SST anomaly vs cell climatology (climatology comes from models/climatology.py)
- Handle missing buoy data (if a buoy has no reading on day D, use next nearest)
- Output columns: buoy_sst_idw, buoy_sst_7d_mean, buoy_sst_30d_mean, buoy_anomaly

features/calcofi_features.py:
- CalCOFI is quarterly, so we need to temporally interpolate to daily
- For each (cell, date), find the nearest CalCOFI station, then linearly interpolate the temperature at 50m and 100m depth across the nearest two cruise dates
- Same for salinity and chlorophyll
- Compute thermocline depth as the depth where dT/dz is maximal in the 0-200m profile
- Output columns: calcofi_temp_50m, calcofi_temp_100m, calcofi_salinity_50m, calcofi_chla, calcofi_thermocline_depth

features/satellite_features.py:
- For each (cell, date), nearest-pixel lookup into the SST raster
- Compute spatial SST gradient (sobel filter on the raster, then lookup)
- DHW value at the cell
- Days since last cold anomaly (cell SST < climatology - 0.5°C)
- Output columns: sat_sst, sat_sst_anomaly, sat_dhw, sat_sst_gradient, sat_days_since_cold

features/build_feature_table.py:
- Orchestrates all three feature modules
- Joins on (cell_id, date)
- Adds day-of-year, month, year columns
- Adds lag features: target (MHW status) lagged 1, 3, 7 days
- Output: parquet table with one row per (cell, date), 2015-2023
- This is the main training-ready table

All functions should be vectorized where possible. Use numpy/pandas, not loops, for the grid operations. Use scipy.spatial.cKDTree for nearest-neighbor lookups.

Write tests that verify:
- Feature table has no duplicated (cell, date) rows
- Every numeric column has <30% missing values
- Lag features are properly shifted (not leaking future info)

Prompt 4: MHW labeling and climatology

Implement the MHW detection logic following Hobday et al. 2016.

models/climatology.py:
- Input: daily satellite SST data, 1983-2012 baseline (or max available)
- For each cell and each day-of-year (1-366), compute:
  - Mean SST using an 11-day centered window (days d-5 to d+5) across all years in baseline
  - 90th percentile SST using the same 11-day window
- Smooth the climatology with a 31-day rolling mean to handle noise
- Output: DataFrame with columns [cell_id, day_of_year, clim_mean, clim_p90]

Note: if you don't have 1983-2012 satellite data available, use the longest baseline you have (e.g., 2014-2019) and document this as a limitation.

models/mhw_labels.py:
- Input: daily SST at each cell + climatology
- For each cell/date, flag MHW-onset if SST > clim_p90 for that day-of-year
- Apply the 5-day rule: a MHW event requires 5+ consecutive days above p90
- Gap rule: 2-day gaps below p90 don't break an event
- Output: binary MHW status per (cell, date)
- Also compute MHW category (Moderate/Strong/Severe/Extreme) based on (SST - clim_mean) / (clim_p90 - clim_mean)

Also implement the label generation for training:
- For each (cell, date), generate targets y_1d, y_3d, y_5d, y_7d indicating MHW status 1/3/5/7 days in the future
- This is the classification target

Write tests using a known Pacific Blob event (July-September 2015 in the California coast) — verify the labels fire during that window.
Prompt 5: LightGBM models
Train LightGBM classifiers for MHW prediction at 4 lead times.

models/lightgbm_model.py:
- Input: feature table from features/build_feature_table.py
- Train/val/test split: train on 2015-2021, validate on 2022, test on 2023
- Important: split by DATE, not randomly. Never leak future into past.
- Train 4 separate LightGBM classifiers, one per lead time (1, 3, 5, 7 days)
- Hyperparameters: num_leaves=63, learning_rate=0.05, n_estimators=500, early_stopping_rounds=50 on val set
- Log all training metrics to MLflow (Databricks-hosted)
- Save trained models to GCS as joblib files

Evaluation:
- Report AUC, PR-AUC, Brier score per lead time on 2023 test set
- Feature importance plots (top 20 features per model)
- Calibration curves

Write a function run_ablation(features_to_drop) that retrains with a feature subset removed. This is how we'll generate the CalCOFI ablation for the Scripps prize — compare full model vs model without calcofi_* features.

Save evaluation results as JSON artifacts for later visualization.
Prompt 6: Raster CNN on Brev.dev
Implement a temporal CNN for SST raster sequences. Target environment: NVIDIA Brev.dev with a single A100 or similar.

models/raster_cnn.py:
- Input: (T=30, H=40, W=50) tensors of daily SST for the SoCal Bight at 5km resolution, preceding 30 days
- Output: per-cell probability of MHW status at T+1, T+3, T+5, T+7 (multi-head output)
- Architecture:
  - 3D conv over (T, H, W) — early layers capture local temporal patterns
  - Spatial attention block
  - Temporal attention over the reduced sequence
  - Four prediction heads (one per lead time)
- Loss: weighted BCE per head, class weights based on MHW prevalence
- Mixed precision training
- Save checkpoints to GCS

Training script should:
- Load feature table and raster data from GCS
- Construct training sequences (rolling 30-day windows)
- Split by date (same as LightGBM)
- Train for max 50 epochs with early stopping
- Log to MLflow
- Output: trained model weights, validation metrics

After training, write a batch inference script that generates per-cell, per-date predictions for the full 2023 test set and saves them as parquet. We'll use this in the ensemble and backtest.

Include a short README section in models/raster_cnn.py explaining how to run this on Brev.dev (which image, which instance type, rough cost/time).
Prompt 7: Ensemble and backtest
Combine the two model families and generate the final backtest.

models/ensemble.py:
- Inputs: LightGBM predictions (4 models, one per lead time) and raster CNN predictions (4 heads)
- For each (cell, date, lead_time), compute ensemble prediction as weighted average: alpha * lgbm_prob + (1-alpha) * cnn_prob
- Tune alpha on the 2022 validation set separately per lead time
- Output: parquet of ensemble predictions for 2023

backtest/evaluate.py:
- Compute on 2023 test set:
  - AUC, PR-AUC, Brier score per lead time (ensemble and individual models)
  - Confusion matrix at p=0.5 threshold
  - Precision at 90% recall
  - Per-event metrics: for each historical MHW event, did we predict it?

Lead time of first alert?
- Generate plots:
  - ROC curves per lead time (seaborn)
  - Calibration plots
  - Time series of predicted vs actual MHW status for 3 example cells
  - Map showing model performance (AUC per cell)
- Save all plots to backtest/figures/ as PNGs
- Save metrics as backtest/metrics.json

Also generate the Pacific Blob replay:
- Rewind to July 1, 2015
- For each day from July 1 to September 30, 2015, get the model's 7-day forward prediction
- Compare against what actually happened
- Save as a parquet that the frontend will animate
- This is the demo money shot

Prompt 8: API and frontend integration

Build the FastAPI backend and prepare for frontend integration.

api/main.py:
- FastAPI app with these endpoints:
  - GET /forecast?lat=&lon=&date= → returns predictions for all 4 lead times with confidence intervals
  - GET /forecast_grid?date=&lead_time= → returns predictions for all cells on a given date (for the heatmap)
  - GET /backtest/blob → returns the 2015 Pacific Blob replay data
  - GET /summary?lat=&lon=&date= → calls Gemini to generate a natural-language summary of the forecast, citing the specific features that drove the prediction (e.g., "elevated CalCOFI 50m temperatures combined with persistent positive SST anomalies")
- CORS enabled for the frontend
- Load models once at startup, cache predictions for common queries
- Deploy target: GCP Cloud Run
- Dockerfile included

For /summary, use the Gemini API:
- Model: gemini-1.5-flash (cheap, fast)
- Prompt template: "Given this marine heatwave forecast for {location} on {date}: MHW probability = {p}, top contributing features = {features}. Generate a 2-3 sentence summary for a fisheries manager. Cite specific oceanographic drivers."
- Return both the raw prediction and the NL summary

Test locally with uvicorn, then write the Cloud Run deploy script.

Prompt 9: Marimo notebook

Write a Marimo notebook (notebooks/analysis.py) that tells the full Tideline story for judges. Structure it like a research paper:

1. Introduction (markdown cell)
   - What are marine heatwaves, why they matter, economic impact

- Citations to Hobday 2016 and relevant MHW literature

2. Data sources (markdown + code cells)
   - Description of NOAA buoys, CalCOFI, satellite SST
   - Load each, show head() and basic statistics
   - Map showing buoy locations, CalCOFI stations, satellite coverage

3. Feature engineering (code cells)
   - Show the unified grid
   - Demonstrate buoy IDW, CalCOFI interpolation, satellite lookup on a single example cell/date
   - Show the final feature table

4. Climatology and MHW detection (code cells)
   - Build the climatology for one cell, plot it
   - Apply Hobday 2016 detection, show MHW events identified in 2015 (Pacific Blob) — this should be visually dramatic

5. Modeling (code cells)
   - Train LightGBM, show feature importance
   - Train raster CNN, show training curves
   - Ensemble, show calibration

6. Ablation study — THE KEY SECTION (code cells)
   - Full model vs no-CalCOFI
   - Report AUC delta
   - Conclude: CalCOFI subsurface data contributes X% AUC lift
   - This is the artifact that wins the Scripps prize

7. Backtest (code cells + plots)
   - 2023 test metrics
   - Pacific Blob replay visualization

8. Discussion and limitations (markdown)
   - What would we do with more time
   - Honest about noise, coverage gaps, climate baseline issues

9. References (markdown)

Make it interactive where possible — Marimo supports UI components, use sliders for the climatology window size, cell selection, etc.
Prompt 10: Sphinx documentation
Set up Sphinx documentation in docs/.

Structure:
- docs/source/conf.py (standard Sphinx config, use sphinx-rtd-theme)
- docs/source/index.rst (landing page)
- docs/source/architecture.rst (system architecture with diagrams)
- docs/source/api.rst (auto-generated from FastAPI using sphinxcontrib-openapi)

- docs/source/data_sources.rst (detailed description of each data source with licenses, citations, update cadence)
- docs/source/model_cards.rst (one card per model: LightGBM, CNN, Ensemble, with intended use, training data, metrics, limitations, ethical considerations following the HuggingFace model card spec)
- docs/source/reproducibility.rst (step-by-step guide to reproducing our results from scratch)

Build the docs with `make html` and include the build output location in the main README.

Add a GitHub Actions workflow that builds docs on push and deploys to GitHub Pages.

A few closing tactical notes

On picking GCP vs AWS. I recommended GCP specifically for the Gemini integration which adds a differentiated demo moment. But if anyone on your team has significantly more AWS experience, go AWS — the hour you save on cloud-service familiarity is worth more than the Gemini hook. You can still use Gemini from AWS (it's just an API).

On the Impulse AI platform. Their prize emphasizes "best models trained on Impulse." Spend 1-2 hours on their onboarding and run at least your LightGBM hyperparameter sweep there. Don't over-invest — if their platform is clunky, cap your time at 3 hours and cut the prize.

On the ablation study. This is the single most important artifact for winning Scripps. If you do nothing else novel, make sure you have a clean, well-visualized comparison of "with CalCOFI features" vs "without." The narrative writes itself: "Scripps' subsurface oceanographic data contributes measurable predictive lift that satellites alone cannot capture."

On team coordination. Designate one person to own the demo script and rehearse it from hour 20 onward. The team that rehearses wins against technically-better teams that don't.

Ready to start building? I'd kick off with Prompt 1 in Claude Code and parallel-start the data downloads with Prompt 2 immediately after. Ping me when you hit your first blocker and I'll help you debug or adapt the plan.

Tab 5

Got it. 13-15 hours is tight but doable for Tideline if you scope ruthlessly. Let me give you the full playbook.

Reality check on scope first

You have three people and ~14 hours. That's ~42 person-hours minus sleep, meals, debugging, and demo prep. Realistic working time is closer to 25-30 person-hours. That means:

- No real-time ingestion. Everything runs on pre-downloaded data.
- No training a temporal transformer from scratch. Gradient-boosted baseline + one deep learning flourish if time permits.
- No production-grade anything. This is a working demo that tells a compelling story.
- Databricks and Brev.dev integration need to be *real* but *minimal* — enough that you can honestly say you used them, not full platform adoption.

With that framing, here's the plan.

Tideline — Full Project Spec

Product Overview

What it is. A forecasting and alerting system for marine heatwaves along the California coast. It ingests historical oceanographic data from NOAA buoys, Scripps CalCOFI surveys, and NOAA satellite sea surface temperature, predicts marine heatwave probability 5-7 days ahead at regional granularity, and presents the forecast through a map-based dashboard with backtest validation on the 2014-2016 Pacific "Blob" and 2019 Northeast Pacific heatwaves.

Who it's for. Three customer segments, in order of TAM:

1. Aquaculture operators (oyster farms, mussel farms, kelp farms) — a 3-day warning lets them harvest early or deploy cooling interventions. California aquaculture is ~$150M annually, US is $1.5B+.
2. Commercial and recreational fisheries — species distribution shifts with temperature, forecasts inform where to fish.
3. Marine conservation orgs and reef managers — triggering protective measures (coral reef monitoring, MPA enforcement intensification).

Why now. Marine heatwaves have tripled in frequency since the 1980s. The 2014-2016 Blob killed an estimated 100 million cod and caused $100M+ in fishery losses. Current NOAA forecasts are global and coarse; regional, actionable forecasts don't exist as a product.

The pitch in one sentence for judges. "We built the equivalent of a hurricane forecast for marine heatwaves — specific enough that an oyster farmer in Humboldt Bay can act on it three days out."

Technical Architecture

```
Data Sources
├── NOAA NDBC Buoy data (sea surface temp, historical)
├── NOAA OISST v2.1 (satellite SST rasters, 0.25° grid, daily)
└── Scripps CalCOFI (cruise-level oceanographic observations)
```

```
        │
        ▼
Ingestion & Storage Layer (AWS S3 + Databricks)
├── Raw zone: parquet files per source
├── Bronze layer: cleaned, schema-enforced
└── Silver layer: feature-engineered, joined across sources
        │
        ▼
Feature Engineering (Databricks notebooks, Delta Lake)
├── Rolling SST anomalies (7d, 30d, 90d)
├── SST gradient features (spatial)
├── Buoy-satellite cross-validation features
└── Heatwave label (NOAA's 90th percentile definition)
        │
        ▼
Model Training (Brev.dev GPU instance)
├── Primary: XGBoost binary classifier (heatwave Y/N at T+5)
├── Registered on Impulse for tracking
└── Backtest on 2014-2016 Blob + 2019 NEP events
        │
        ▼
Inference API (AWS Lambda + API Gateway)
├── GET /forecast?lat=X&lon=Y&horizon=5
└── GET /backtest?event=blob2015
        │
        ▼
Dashboard (React, deployed on AWS Amplify or Vercel)
├── Leaflet map with heatwave probability overlay
├── Time slider (historical backtest mode)
├── Regional drill-down with confidence intervals
└── "Alert subscription" mock form for the pitch
        │
        ▼
Reproducibility Layer
├── Marimo notebook: end-to-end analysis + model training
└── Sphinx site: methodology, data sources, API docs
```

What Makes This Non-Trivial

1. **Multi-source data fusion at different resolutions.** Buoys are point observations, OISST is a 0.25° grid, CalCOFI is irregular cruise transects. Interpolating and aligning these onto a common space-time grid is real work.

2. **Proper temporal validation.** You cannot random-split time series data. Backtest has to respect temporal ordering and ideally use walk-forward validation. Judges who know ML will ask about this.

3. **Heatwave definition is non-obvious.** The NOAA standard uses Hobday et al. 2016 — a marine heatwave is when SST exceeds the 90th percentile of a 30-year climatological baseline for at least 5

consecutive days. Computing this correctly is the difference between a real project and a toy.

4. **Confidence intervals on predictions.** Quantile regression or conformal prediction makes the forecast actionable. "70% probability of heatwave with 60-80% CI" is useful; a point prediction is not.

Prize Stacking Strategy

Concrete mapping of what each sponsor will see:

Prize	What you show them
Scripps Data	CalCOFI integration is front and center; the Marimo notebook walks through Scripps data specifically
AWS Cloud Platform	S3 for data lake, Lambda for inference API, API Gateway, Amplify for hosting — architecture diagram with AWS services labeled
Databricks	Delta Lake bronze/silver/gold architecture, notebooks for feature engineering, lineage visible
Impulse AI	Model training and versioning runs through Impulse; show the experiment dashboard
NVIDIA Brev.dev	Training job runs on Brev.dev GPU instance (even a modest one) — screenshot the instance in your demo
Marimo/Sphinx	Full analysis notebook in Marimo, documentation site in Sphinx
IEEE Info Theory	Frame the multi-source fusion as signal extraction; one slide on KL divergence between buoy and satellite distributions
Scripps General Challenge	The overall project is the submission here
Entrepreneurship (Innovative Idea)	Pitch deck: aquaculture TAM, fisheries angle, climate tie-in

Realistic target: win 3-5 of these. Even winning 2 plus placing in General Challenge is a strong result.

Team Role Assignment

Tanmay — Data & ML

- Data ingestion, Databricks pipeline, feature engineering
- Model training on Brev.dev
- Backtest and evaluation
- Marimo notebook

Ashwin — Cloud Infra & API

- AWS S3 setup, Lambda inference function, API Gateway

- CI/CD for dashboard deployment
- Sphinx documentation site
- Architecture diagram

JD — Frontend & Product

- React dashboard with map + time slider
- Demo script and video
- Pitch deck for entrepreneurship track
- Cross-examination prep ("why not just use NOAA's existing forecast")

This plays to each of your strengths based on the project history.

Hour-by-Hour Build Plan

T+0:00 to T+1:00 (5:30-6:30 PM) — Setup sprint, no coding yet

- All three on a call. Confirm roles, set up shared repo, Slack/Discord channel.
- Ashwin: create AWS account or use existing, set up S3 bucket, Brev.dev instance, Databricks workspace (free trial if needed), Impulse account.
- Tanmay: download datasets locally first as insurance policy (see data acquisition section below).
- JD: scaffold the React app, get Leaflet working with a placeholder map, start pitch deck outline.

T+1:00 to T+4:00 (6:30-9:30 PM) — Data in, baseline features

- Tanmay: land raw data in S3 and Databricks. Get bronze layer working. Compute SST climatology (90th percentile baseline). Generate heatwave labels for California coastal region, 2012-2020 date range.
- Ashwin: stub out the Lambda function with a hardcoded response so the frontend can integrate before the real model exists.
- JD: integrate a mock forecast API, get the map overlay working with fake probability values to prove the pipeline end-to-end.

T+4:00 to T+7:00 (9:30 PM - 12:30 AM) — First model

- Tanmay: feature engineering in Databricks notebook. Train XGBoost baseline on Brev.dev. Target: AUC > 0.75 on walk-forward backtest.
- Ashwin: wire real model predictions into Lambda. Set up API Gateway. Write Sphinx scaffolding.
- JD: dashboard polish. Time slider for historical backtest. Regional drill-down.

T+7:00 to T+9:00 (12:30 AM - 2:30 AM) — Integration and hardening

- Full end-to-end test. Fix whatever's broken.
- Tanmay: if baseline works, add quantile regression for confidence intervals. If not, stick with point prediction.
- Ashwin: record Impulse experiments, document the Databricks pipeline.
- JD: lock in the Blob 2015 backtest as the headline demo moment.

T+9:00 to T+12:00 (2:30 AM - 5:30 AM) — Sleep rotation

- Whoever is freshest continues polishing. Others sleep 2-3 hours minimum. Do not skip sleep entirely.

T+12:00 to T+14:00 (5:30 AM - 7:30 AM) — Polish

- Marimo notebook finalization (Tanmay).
- Sphinx docs finalization (Ashwin).
- Demo video recording (JD).
- Pitch deck complete (JD with team input).

T+14:00 to T+15:30 (7:30 AM - 9:00 AM) — Dry runs

- Three full dry runs of the demo. Time them. Cut whatever makes you go over 2 minutes.
- Stress-test the cross-examination questions (list below).

T+15:30 to T+19:30 (9:00 AM - 1:00 PM) — Buffer + submission

- Fix anything broken. Submit to Devpost well before 1 PM.
- Do not touch code after 11 AM. Final hour is for submission logistics only.

Data Acquisition (do this first, insurance policy)

Download these to local disk *before* touching cloud:

1. **NOAA OISST v2.1** — https://www.ncei.noaa.gov/products/optimum-interpolation-sst — daily 0.25° global SST, pull 2012-2020 for lat 32-42°N, lon 235-245°E (California coast).
2. **NOAA NDBC buoys** — https://www.ndbc.noaa.gov/ — pick 5-8 California buoys (e.g., 46013, 46014, 46028, 46042). Download historical standard meteorological data.
3. **Scripps CalCOFI** — https://calcofi.org/data/ — bottle database, surface temp measurements.

Total download is probably 2-5 GB. Do this first so if APIs fail you're not stuck.

Structured Prompts for Coding IDEs

These are written for Claude Code / Cursor. Each prompt is self-contained and assumes an empty working directory. Feed them one at a time, verify output, then continue.

Prompt 1: Repository Scaffolding

I'm building a marine heatwave forecasting system called Tideline for a hackathon.
I need you to set up the repository structure.

Create the following directory structure with placeholder files:

```
tideline/
├── README.md              # Project overview, team, pitch
├── .gitignore             # Python, Node, env files, data/, models/
├── data/                  # Local data cache (gitignored)
│   ├── raw/
```

```
│   ├── bronze/
│   └── silver/
├── infra/
│   ├── aws/
│   │   ├── lambda_handler.py    # Inference API handler
│   │   └── requirements.txt
│   └── architecture.md          # Architecture diagram (mermaid)
├── pipeline/
│   ├── ingest_oisst.py          # NOAA satellite SST ingestion
│   ├── ingest_buoys.py          # NDBC buoy data
│   ├── ingest_calcofi.py        # Scripps CalCOFI data
│   ├── features.py              # Feature engineering
│   ├── labels.py                # Heatwave labeling (Hobday et al. 2016)
│   └── train.py                 # XGBoost training
├── notebooks/
│   └── tideline_analysis.py     # Marimo notebook
├── dashboard/                   # React app (Vite + TypeScript)
│   ├── package.json
│   ├── vite.config.ts
│   ├── src/
│   │   ├── App.tsx
│   │   ├── components/
│   │   │   ├── ForecastMap.tsx
│   │   │   ├── TimeSlider.tsx
│   │   │   └── RegionDetail.tsx
│   │   └── api/
│   │       └── tideline.ts      # API client
├── docs/                        # Sphinx documentation
│   ├── conf.py
│   └── index.rst
└── demo/
    ├── script.md                # Demo script
    └── pitch_deck.md            # Pitch outline
```

Write a README.md that covers: what Tideline is, the three customer segments (aquaculture, fisheries, conservation), the tech stack, and how to run locally. Keep it under 200 lines. Include an architecture mermaid diagram.

Use Python 3.11, Node 20, TypeScript. pyproject.toml for Python packaging with these dependencies: xarray, netcdf4, pandas, numpy, scikit-learn, xgboost, boto3, fastapi, uvicorn, marimo.

Prompt 2: Data Ingestion

In the pipeline/ directory, implement three data ingestion scripts for the Tideline marine heatwave project.

Target region: California coast. Lat 32 to 42 N, Lon 235 to 245 E (equivalent to -125 to -115 W). Date range: 2012-01-01 to 2020-12-31.

1. ingest_oisst.py
   - Download NOAA OISST v2.1 daily data from NOAA NCEI or use xarray with OpenDAP: https://www.ncei.noaa.gov/thredds/dodsC/OisstBase/NetCDF/V2.1/AVHRR/
   - Subset to California region
   - Save as parquet in data/raw/oisst/ partitioned by year
   - Each row: date, lat, lon, sst_celsius
   - Handle network failures with retry
   - Progress bar with tqdm
   - If online download fails, fall back to reading pre-downloaded netCDF files from data/raw/oisst_local/

2. ingest_buoys.py
   - Pull historical standard meteorological data for NDBC buoys: 46013, 46014, 46028, 46042, 46054, 46011
   - URL pattern:
https://www.ndbc.noaa.gov/view_text_file.php?filename={buoy}h{year}.txt.gz&dir=data/historical/stdmet/
   - Parse the fixed-width format (WD, WSPD, GST, WVHT, DPD, APD, MWD, PRES, ATMP, WTMP, DEWP, VIS, TIDE)
   - Extract WTMP (water temp) specifically
   - Save as parquet in data/raw/buoys/
   - Each row: timestamp, buoy_id, lat, lon, water_temp_celsius

3. ingest_calcofi.py
   - Download CalCOFI bottle database from https://calcofi.org/data/oceanographic-data/bottle-database/
   - Direct CSV link if available, otherwise document manual download step
   - Filter to 2012-2020, surface measurements (depth < 10m)
   - Save as parquet in data/raw/calcofi/
   - Each row: timestamp, lat, lon, temperature, salinity, cruise_id

All three scripts should:
- Accept CLI args for date range and output path
- Log progress to stdout
- Write a summary JSON at the end (rows written, date range, any errors)
- Be idempotent (rerunning doesn't duplicate data)

After all three scripts, write a pipeline/validate_ingestion.py that:
- Reads the parquets
- Prints row counts, date ranges, coverage statistics
- Asserts minimum thresholds (>1M rows for OISST, >100k for buoys, >1k for CalCOFI)

Prompt 3: Feature Engineering and Labeling

Implement pipeline/labels.py and pipeline/features.py for the Tideline project.

labels.py — Marine heatwave labeling following Hobday et al. 2016:
- Compute daily climatology for each grid cell: mean and 90th percentile of SST over a 30-day centered window across all years 2012-2020 (use 2012-2017 as baseline to avoid leakage into test years 2018-2020)
- A marine heatwave is when SST exceeds the 90th percentile climatology for at least 5 consecutive days

- Output: for each (date, lat, lon), a label `mhw_active` (0 or 1) and
  `mhw_will_occur_T+N` for N in {1, 3, 5, 7} days ahead
- Save to data/silver/labels/ as parquet

features.py — Feature engineering:
- Input: the bronze OISST, buoy, and CalCOFI tables
- Output: a unified silver table keyed by (date, lat, lon) on the OISST grid
- Features to compute:
  * sst_anomaly_7d: current SST minus 7-day trailing mean
  * sst_anomaly_30d: current SST minus 30-day trailing mean
  * sst_trend_7d: linear trend over past 7 days
  * sst_spatial_gradient_ns: north-south SST gradient
  * sst_spatial_gradient_ew: east-west SST gradient
  * nearest_buoy_temp: interpolated buoy water temp (nearest in space+time within 48h)
  * nearest_buoy_distance_km: sanity feature
  * calcofi_recent_temp: most recent CalCOFI surface temp within 30 days and 100km
  * month_sin, month_cos: cyclical encoding
  * year: for drift detection
- Handle missing values: forward fill within 3 days, then impute with climatology
- Save to data/silver/features/ partitioned by year

Both should run on pandas/dask on a single machine (no Spark dependency).
Use xarray for the OISST grid operations. Include unit tests in pipeline/tests/
for the climatology computation and heatwave labeling against 1-2 known
historical heatwave events (2014-09-01 shows elevated SST, 2015-05-01 shows
Blob in full force).

Prompt 4: Model Training

Implement pipeline/train.py for the Tideline marine heatwave forecasting model.

Task: binary classification, predict whether a marine heatwave will be active
5 days ahead given current features.

Requirements:
- Load silver features and labels
- Train/validation/test split using walk-forward by time:
  * Train: 2012-2017
  * Validation: 2018
  * Test: 2019-2020 (includes the Northeast Pacific 2019 heatwave)
- Primary model: XGBoost binary classifier
  * Use scale_pos_weight to handle class imbalance
  * Early stopping on validation AUC
  * Hyperparams: max_depth=6, learning_rate=0.05, n_estimators up to 1000,
    subsample=0.8, colsample_bytree=0.8
- Secondary model: quantile regression on SST anomaly (for confidence intervals)
  — optional, do this only if XGBoost baseline completes with AUC > 0.75
- Evaluation:
  * AUC, precision@top-10%, recall at 0.5 threshold
  * Per-year metrics (critical for showing it works on held-out years)

  * Calibration plot (reliability diagram)
  * Feature importance (SHAP if time permits, gain if not)
- Save model to models/xgboost_t5.pkl
- Save evaluation report to models/eval_report.json with all metrics

Integrate with Impulse AI for experiment tracking:
- Use the Impulse SDK (check their docs — if unavailable, fall back to MLflow and note this as a backup)
- Log: hyperparameters, metrics, feature importance, model artifact

Training should run on a Brev.dev GPU instance. Add a README in the repo documenting:
- How to launch a Brev.dev instance (t4 or a10g should be plenty)
- Environment setup (pip install requirements)
- Command to run training
- Expected runtime (~15-30 minutes)

Include a notebooks/tideline_analysis.py Marimo notebook that reproduces the full analysis: loads features, shows EDA plots, runs a mini training loop, visualizes the 2015 Blob backtest. This is the reproducibility artifact for the Marimo prize track.

Prompt 5: AWS Inference API

Implement the AWS inference API for Tideline.

File: infra/aws/lambda_handler.py

Endpoints (via API Gateway REST API):
1. GET /forecast?lat={lat}&lon={lon}&horizon={days}
   - Returns: { "probability": 0.67, "ci_low": 0.55, "ci_high": 0.78,
     "forecast_date": "2020-06-15", "horizon_days": 5 }

2. GET /backtest?event={event_id}&date={date}
   - Event IDs: "blob2015", "nep2019"
   - Returns a grid of predictions for that event date for the California region
   - Format: { "date": "2015-09-01", "grid": [{"lat": 36.5, "lon": -122.5,
     "probability": 0.84, "actual": 1}, ...] }

3. GET /health
   - Sanity check

Implementation:
- Model loaded from S3 at cold start (bucket: tideline-models, key: xgboost_t5.pkl)
- Features for any (lat, lon, date) query computed from pre-cached silver data stored in S3 (bucket: tideline-silver, prefix: features/)
- Response time target: < 2 seconds warm, < 8 seconds cold
- Use AWS Lambda Python 3.11 runtime
- Package size: keep under 50 MB (xgboost + pandas + pyarrow)
- Use Lambda layers for large dependencies if needed

Deployment:
- Terraform or AWS SAM template in infra/aws/template.yaml
- Define: Lambda function, API Gateway REST API, S3 buckets, IAM role
- Environment variables: MODEL_BUCKET, MODEL_KEY, FEATURES_BUCKET
- CORS enabled for the dashboard origin

Write infra/aws/deploy.sh that:
1. Builds the Lambda package
2. Uploads to S3
3. Deploys SAM stack
4. Prints the API Gateway URL

Include a test script infra/aws/test_api.sh with 5 curl commands hitting different endpoints and verifying responses.

Prompt 6: React Dashboard

Build the Tideline dashboard in dashboard/ using Vite + React + TypeScript.

Dependencies:
- react-leaflet for the map
- leaflet.heat for heatmap overlay
- recharts for charts
- tailwindcss for styling
- date-fns for date handling

Layout:
- Header: "Tideline — Marine Heatwave Forecast" logo/title, current forecast date selector
- Main view: split 70/30
  * Left 70%: Leaflet map of California coast (default zoom to lat 32-42, lon -125 to -115) with color-coded heatwave probability overlay (green to red gradient, 0 to 1)
  * Right 30%:
    - "Currently viewing" panel with date, mode (live/backtest), and region
    - Forecast chart: probability over next 7 days for selected region
    - Alert thresholds panel (mock subscription form)
- Footer: time slider with two modes
  * "Live forecast" mode: slider for horizon (1-7 days ahead)
  * "Historical backtest" mode: slider over the 2015 Blob event (2015-05-01 to 2015-12-31) with a toggle showing model prediction vs actual heatwave extent

Components:
- ForecastMap.tsx: Leaflet map with probability overlay, click-to-select region
- TimeSlider.tsx: dual-mode time slider
- RegionDetail.tsx: right-side panel
- AlertSubscribe.tsx: mock form, just captures email + region (for the pitch)
- ModeToggle.tsx: switch between live and backtest

API client in src/api/tideline.ts:
- fetchForecast(lat, lon, horizon) — calls /forecast
- fetchBacktest(event, date) — calls /backtest
- Env var VITE_API_URL for the API Gateway endpoint
- Hardcoded fallback data in src/api/fallback.ts so the dashboard works offline during the demo if AWS goes down. This is critical — the demo must not depend on live API.

Polish:
- Loading states
- Empty state when region has no data
- A "?" button that opens a modal explaining the Hobday heatwave definition (for judges who ask)
- Keyboard shortcuts: arrow keys on time slider, space to toggle mode

Deploy to Vercel. Include vercel.json with env var placeholders.

Prompt 7: Sphinx Documentation

Set up Sphinx documentation in docs/ for the Tideline project.

Use the Furo theme for a modern look. Pages:

1. index.rst — Landing page
   - Project overview
   - Architecture diagram (embed mermaid via sphinxcontrib-mermaid)
   - Links to other pages

2. methodology.rst — The scientific approach
   - Marine heatwave definition (Hobday et al. 2016, cite properly)
   - Data sources: NOAA OISST, NDBC buoys, Scripps CalCOFI
   - Feature engineering rationale
   - Model choice justification
   - Validation strategy (walk-forward, why not random split)

3. api.rst — API reference
   - Auto-generated from the FastAPI/Lambda handler using sphinx-autodoc
   - Document each endpoint with request/response examples

4. data_sources.rst — Data provenance
   - For each source: what it is, coverage, license, access method, refresh cadence
   - Highlight the CalCOFI integration (for Scripps judges)

5. deployment.rst — How to deploy
   - AWS setup
   - Databricks pipeline
   - Brev.dev training
   - Impulse integration

6. evaluation.rst — Model performance
   - Auto-populated from models/eval_report.json
   - AUC, precision, recall, calibration plot
   - Backtest results on 2015 Blob and 2019 NEP events

conf.py should include:
- Extensions: autodoc, napoleon, mermaid, copybutton
- Theme: furo
- Project metadata

Write a docs/Makefile and build instructions. Deploy the built HTML to GitHub Pages via a workflow file (.github/workflows/docs.yml).

Prompt 8: Demo Polish and Pitch

In demo/, produce two artifacts.

demo/script.md — 2-minute demo script with exact beats and time markers:

[0:00-0:15] Hook
- "Marine heatwaves killed 100 million Pacific cod in 2016 alone. Current forecasts are global and coarse. We built Tideline: hurricane-style forecasting for marine heatwaves."

[0:15-0:45] Problem + solution
- Show a map of the 2015 Blob
- "Here's what the 2015 Pacific Blob looked like. Fisheries lost over $100M. No one was warned with the specificity or lead time they needed."
- Show the Tideline dashboard pointed at 2015-05-01
- "This is our model's forecast, trained only on data up to 2014. We'd have called the Blob three days ahead at regional granularity."

[0:45-1:30] Product walkthrough
- Live-ish dashboard interaction
- Click on Humboldt Bay
- Show 7-day probability forecast with confidence bands
- Show alert subscription form
- "An oyster farmer here gets a Slack alert at 70% probability 3 days out. Enough to harvest early or deploy mitigation."

[1:30-2:00] The business
- Three segments: aquaculture ($1.5B US), fisheries, conservation
- Go-to-market: start with aquaculture insurance partnerships (their loss ratios tie directly to MHW frequency)
- Tech stack callout: "Built on AWS, Databricks, NVIDIA Brev.dev, trained with Impulse, real Scripps CalCOFI data in every prediction."

demo/pitch_deck.md — 8-slide deck in markdown (convert to Google Slides manually):
1. Title: Tideline, team names, one-liner

2. Problem: marine heatwave cost data, 2014-2016 Blob visual
3. Why now: frequency has tripled since 1980s (cite)
4. Product: dashboard screenshot + 3 customer segments
5. Technical approach: architecture diagram
6. Proof: backtest results on Blob + NEP events (AUC, precision)
7. Market: TAM breakdown, GTM (aquaculture insurance first)
8. Ask: what we'd do with $X (data partnerships with NOAA/Scripps, sales hire for aquaculture channel)

Also generate a README section "Cross-examination prep" with answers to these questions:
1. Why can't I just use NOAA's existing MHW tracker?
2. How do you know your model isn't overfitting?
3. What's your accuracy on the 2019 NEP event specifically?
4. Who's your first paying customer?
5. What's the unit economics of an aquaculture SaaS subscription?
6. Why is this a software problem, not a sensor/buoy problem?
7. How do you handle regime shifts from climate change?
8. What happens when your model is wrong and a farmer over-harvests?

Answer each in 2-3 sentences. Honest answers, not defensive ones.

Cross-Examination Prep (read this now)

The judges and ChatGPT/Gemini/Grok review will ask these. Have answers ready:

1. **"NOAA already has a MHW Tracker, why do I need you?"** NOAA's tracker is retrospective and global at 0.25° resolution. Tideline is forecast-forward (5-7 days) and integrates local data (buoys, CalCOFI) that the global tracker doesn't use. We also deliver alerts; NOAA is a dashboard researchers check.

2. **"Is your backtest real or did you leak data?"** Walk-forward split. Train on 2012-2017, validate on 2018, test on 2019-2020. Climatology baseline computed only on training years. The 2019 NEP heatwave is a legitimate held-out test and we'll report per-year metrics.

3. **"How's this different from a weather forecast?"** Ocean has much longer thermal inertia than atmosphere. MHW forecasting at 5-7 day horizon is actually more tractable than weather at that range, which means a well-designed statistical model can be genuinely useful.

4. **"Who pays for this?"** First customer archetype: aquaculture insurance provider. MHW frequency is in their loss models but they don't have operational forecasts. A $25k/year SaaS license to feed their underwriting is a natural first check. From there, direct sales to large aquaculture operations (Pacific Seafood, Hog Island Oyster Co).

5. **"Why didn't you use a fancy deep learning model?"** We did a rigorous baseline first. XGBoost with proper feature engineering beats poorly-tuned transformers on tabular time series. If we had three weeks we'd try a ConvLSTM on the SST raster directly. In 15 hours we prioritized a correctly validated

baseline over a cool architecture.

Risk Register and Mitigations

Risk	Probability	Mitigation
OISST download fails or is slow	High	Pre-download to local disk in hour 0 as insurance
Databricks free trial limitations	Medium	Do feature engineering in pandas locally, move to Databricks only for showcase
Brev.dev instance issues	Medium	Train locally as fallback, just screenshot the Brev.dev instance for the demo
Model performs badly (AUC < 0.7)	Medium	Pivot pitch to "early warning system" rather than "forecast" — still useful at lower performance
AWS Lambda cold starts hurt demo	Low	Pre-warm before demo, have fallback hardcoded responses in frontend
Impulse API doesn't work as expected	Medium	Fall back to MLflow locally, note in docs that production would use Impulse
One team member gets stuck on a task	High	Time-box every task at 2x estimate, have 1-on-1 check-ins every 3 hours

---

Final strategic advice

A few things from watching hackathons go sideways:

Ship the skeleton first. By hour 3 you should have a dashboard talking to an API returning fake data from a Lambda. Don't let any one person disappear into their piece until the end-to-end skeleton exists. Integration debugging at hour 13 is how projects die.

The backtest is the demo, not the live forecast. Nobody can verify a live forecast at a hackathon. The 2015 Blob backtest is falsifiable and dramatic — your model, trained only on 2012-2014 data, calls a historical disaster three days out. That's the moment judges remember.

Don't spread across sponsors too thin. Pick your top 4 target prizes and make sure each has a screenshot-worthy moment in the demo: Scripps (CalCOFI explicitly labeled), AWS (architecture diagram), Databricks (the pipeline), Impulse (experiment dashboard). Give each judge something to point at. The rest are upside.

Start the pitch deck in hour 2, not hour 14. JD should be iterating on the pitch deck throughout. Product/Entrepreneurship track is won by narrative, not technical depth.

Good luck. Ping me if you hit specific snags during the build — I can help debug, rewrite a prompt, or rework a piece of the architecture in real time.

PMF notes

What is the problem?
San Diego aquaculture operators and reef conservation groups face growing risks from marine heatwaves in the Southern California Bight, where warmer waters exacerbate viral outbreaks and kelp die-offs.

Details behind the problem

- Aquaculture faces OsHV-1 virus outbreaks in oysters, spiking at 21-24°C during MHWs, causing mass die-offs without early warnings—*current thresholds are reactive, not predictive*.
- Kelp restoration suffers delayed recovery after events like 2014-2016 Blob, as heatwaves degrade forests before interventions; broad NOAA tracking misses local granularity.

What is a high overview of what we will do?
Tideline addresses this by fusing buoy, CalCOFI, and satellite data for 5-7 day MHW forecasts at coastal granularity.

Who is involved/faces the problem directly?
Port of San Diego aquaculture operators (e.g., San Diego Bay Aquaculture) and kelp restoration groups like Dr. Jennifer Smith's lab at Scripps/UCSD. These entities operate in San Diego Bay and nearby coasts, directly impacted by local MHWs.

How is this different from what already exists and what is being used?

- Port operators rely on NOAA's real-time buoy data and the experimental "Blobtracker" (California Current MHW Tracker), which detects ongoing heatwaves using SST anomalies but offers no lead-time predictions.
- Scripps/UCSD kelp researchers use satellite SST and historical CalCOFI profiles for post-event analysis, lacking fused, forward-looking models.

Why Tideline wins
Tideline fuses disparate data (hourly buoys, quarterly CalCOFI, daily satellites) into a 10km-grid forecast with probabilities and confidence intervals, outperforming Blobtracker's detection-only by providing actionable 5-7 day leads via LightGBM and CNN ensemble. Backtests on 2015 Blob prove 70-80% AUC-ROC; auditable Marimo notebook and API enable quick integration.

Aspect	Current Tools (NOAA Blobtracker/SST)	Tideline
Lead Time	None (real-time tracking) `integratedecosystemasses`	5-7 days [project spec]
Resolution	1/4° global grid `integratedecosystemasses`	10km coastal Bight
Fusion	Single-source SST `integratedecosystemasses`	Buoy + CalCOFI + satellite
Output	Anomaly maps `integratedecosystemasses`	Prob maps + CIs + API

Why should people invest?
Operators save $100K+ per outbreak via early harvest/move (e.g., OsHV-1 mitigation); kelp projects cut restoration costs 20-30% with preemptive seeding. $500/month subscription per dashboard user yields ROI in one event; Scripps ties boost credibility for grants.

Due Diligence

Good. The mentor is right, and she's given you the single most valuable piece of hackathon advice: **start narrow, prove it works, show the wedge, then extrapolate.** Most teams pitch TAM from day one; the ones that win pitch a beachhead.

Let me put on the VC hats and run this through real due diligence, then rebuild the pitch with the narrowing she suggested.

The VC Panel — 5 Perspectives

VC 1: The Climate Tech Generalist (thinks like Lowercarbon Capital)

What they like. Climate adaptation (not mitigation) is underinvested relative to mitigation. Marine heatwaves are a concrete, physical, worsening problem with documented economic damage. The 2014-2016 Blob and 2023 North Atlantic heatwave created insurance claims and aquaculture losses in the billions. Forecasting tools for climate adaptation are a category they're actively hunting.

What concerns them. Forecasting is a feature, not a company. NOAA could ship this. Academic labs could ship this for free. The moat has to be something other than the model.

Their key question. "Why does this become a company and not a NOAA research product?"

VC 2: The Vertical SaaS Investor (thinks like Bessemer, Altos)

What they like. Aquaculture is a real vertical with real buyers. It's fragmented, digitally underserved, and has hard operational constraints that software can address. $500/month per operator with 10,000+ US aquaculture operations and 50,000+ globally is a real ARR story.

What concerns them. Aquaculture operators are notoriously slow to adopt software. Sales cycles are long, checks are small, and churn is high if the tool doesn't prevent the next disaster. The unit economics need to survive a 2-year payback cycle.

Their key question. "Walk me through the sales motion for the first 10 customers. Who do you call Monday morning?"

VC 3: The Enterprise / Insurance Investor (thinks like Anthemis, MunichRe Ventures)

What they like. The real money in climate adaptation forecasting is not selling dashboards to farmers. It's selling data products to insurers, reinsurers, and commodity traders who already pay seven figures for weather data from IBM/The Weather Company. Marine heatwave forecasts with lead time become a parametric insurance trigger.

What concerns them. The team is undergrads with no insurance domain experience. The go-to-market to insurers takes 18-24 months; the team won't survive that runway.

Their key question. "What's the product for a parametric insurer and can you show me the data you'd sell them?"

VC 4: The Data Infrastructure Investor (thinks like Amplify Partners)

What they like. Multi-source environmental data fusion is a hard, defensible infrastructure problem. If Tideline becomes the canonical source of regional MHW forecasts, it's a pick-and-shovel business serving downstream applications (aquaculture, insurance, conservation, fisheries management).

What concerns them. Data infrastructure companies need a distribution advantage, not just a technical one. Why does anyone route their MHW data through Tideline instead of directly ingesting NOAA?

Their key question. "What's the data asset you're building that gets more valuable over time?"

VC 5: The Skeptic (thinks like a hackathon judge who's seen 40 pitches)

What they notice immediately.

- Oyster farming margins are thin. Is the $100K savings claim per outbreak real or theoretical?
- Kelp restoration groups are grant-funded nonprofits. They are not a commercial customer; they are a validation partner dressed up as one.
- "San Diego Bay Aquaculture" — did you actually talk to them? Because if you pitch this to judges and one of them is from Scripps, they will know.
- 70-80% AUC on a single backtest is not "proven." It's a data point.

Their key question. "Did you interview a single real aquaculture operator, or are you inferring their problem from papers?"

Due Diligence: Strengths, Weaknesses, Fixes

Strengths (lean into these)

1. **The OsHV-1 / 21-24°C threshold is a genuinely great wedge.** This is specific, physiologically real, and ties a forecast value directly to an operational action. This is your whole pitch.

2. **San Diego geographic focus is correct.** Concentrated aquaculture, active Scripps collaboration, well-instrumented coastline. Best possible starting geography in North America for this product.

3. **Scripps proximity is a real moat.** You go to UC Santa Cruz, you're building on Scripps data, Jennifer Smith's lab is a warm intro. That's a credibility and data access story that generic climate-tech startups can't replicate.

4. **The timing narrative is real.** 2023 was the warmest ocean year on record. MHW frequency and severity are accelerating. Adaptation tools are moving from nice-to-have to required.

Weaknesses and how to fix them in the pitch

Weakness 1: "Why not NOAA?" *Fix.* NOAA's Blobtracker is detection, not forecast. NOAA has a research mandate, not a customer-success mandate. An oyster farmer in Carlsbad cannot call NOAA when OsHV-1 risk rises. Tideline is what you use *after* NOAA tells you the heatwave is already here. Frame it as "operational layer on top of public science infrastructure," not "competitor to NOAA."

Weakness 2: "$100K savings is unverified." *Fix.* Anchor the number to a published incident. The 2018-2019 Morro Bay OsHV-1 outbreak documented specific oyster mortality. Translate mortality × market price × acreage into a per-operator loss range. Cite it. Now the $100K is "within the range of the 2019 Morro Bay event, scaled to a mid-sized operator." Defensible.

Weakness 3: "Kelp groups are not customers." *Fix.* Reposition them. Kelp restoration groups are not a revenue segment; they are a **credibility and data-partnership segment**. You give them the tool free or cheap, you get co-authored case studies and Scripps endorsement. That's the hackathon pitch: "Paid customers are aquaculture operators. Our research partners are Scripps-affiliated kelp labs." Two roles, cleanly separated.

Weakness 4: "$500/month subscription is too cheap to matter and too expensive for a small farm." *Fix.* Tier it. Single-farm tier at $300/month. Multi-site operators at $1,500-5,000/month. Insurance/enterprise data feed at $10k+/month. The hackathon pitch leads with the aquaculture tier because it's the beachhead, then mentions the enterprise tier as the expansion motion. This is standard vertical SaaS economics and VCs will recognize the shape immediately.

Weakness 5: "You're undergrads, insurance is 18 months away." *Fix.* You don't pitch insurance as the Year 1 plan. You pitch the beachhead, then say "Year 2 expansion is parametric insurance data products — we're already being asked about this by X." Even if you don't have that conversation yet, the point is to show you understand the expansion path, not to commit to it.

Weakness 6: "Did you talk to anyone?" *Fix.* You need to do ONE customer interview before the pitch. One phone call, one email thread, anything. Even a reply from a shellfish growers' association listing their top operational concerns. The pitch goes from "we believe operators need this" to "we spoke to [person/org], they told us X." This is the single highest-leverage 30 minutes you can spend tonight.

The Narrowed Pitch (per your mentor)

The Wedge: Carlsbad Aquafarm + OsHV-1

One specific operator. One specific threat. One specific action the tool enables.

> **Carlsbad Aquafarm** is a Pacific oyster operation on the San Diego coast. When ocean temperatures sustain 21-24°C during a marine heatwave, Ostreid Herpesvirus-1 (OsHV-1) can wipe out their spat (juvenile oysters) in 48-72 hours. There is no treatment. The only intervention is early harvest of market-size stock and relocation of juveniles to deeper, cooler water — but both require 3+ days of lead time.
>
> Today, they rely on NOAA buoy readings and the Blobtracker dashboard, both of which tell them a heatwave is already happening. By then, it's too late.
>
> Tideline forecasts marine heatwave probability 5-7 days out at 10km coastal granularity. Integrated with the OsHV-1 thermal threshold, it produces an operational risk score that tells Carlsbad Aquafarm on Monday that Thursday's conditions will likely exceed the virus activation threshold — giving them the 72 hours they need to act.

This is the pitch. It's one customer, one threat, one action, one number (72 hours of lead time) that connects science to dollars.

The Extrapolation

Once you've told that story, extrapolation is effortless:

- Same model, different operator → all ~40 commercial shellfish operations along the California coast
- Same model, different threat → kelp forest degradation (your Scripps partnership segment)
- Same data infrastructure, different customer → parametric insurance data feed (Year 2)
- Same approach, different geography → Pacific Northwest, New England, Chilean salmon, Norwegian aquaculture

You don't have to build any of these to pitch them. You just have to show that the thing you built for Carlsbad generalizes cleanly.

The Market Sizing, Honestly

Hackathon judges see made-up TAMs all day. Do this instead.

Beachhead (what you can reach in Year 1): California shellfish aquaculture. ~40 commercial operators. Conservative ACV $6k/year (the $500/mo tier). $240k ARR ceiling in Year 1.

Near-expansion (Year 2): West Coast shellfish + kelp partnerships + first insurance pilot. ~200 operators + 1-2 insurance data contracts at $50k each. $1.5-2M ARR.

Horizon (Year 3-5): Global aquaculture forecasting platform + parametric insurance data infrastructure. Global aquaculture is $300B+ industry; even 0.01% capture through SaaS is $30M ARR. Parametric climate insurance is a $20B+ market growing 20%+ annually.

This is the shape VCs want to see: a small, reachable Year 1, a credible Year 2 expansion, and an honest (not absurd) long-term ceiling.

Unit Economics (two-line version for the pitch)

> "A single prevented OsHV-1 event at Carlsbad pays for 15+ years of Tideline subscription. Our job is to be right once."

That's the whole unit economics slide. Judges will remember this.

The New Pitch Deck (8 slides, narrowed)

Slide 1 — Title Tideline. Marine heatwave forecasting for coastal aquaculture. Team names. Built on Scripps data.

Slide 2 — The Wedge Photo of a Carlsbad oyster farm. One line: "When ocean temps hit 21°C, this farm has 48 hours before a virus can wipe out their crop. Today, they find out when it's already happening."

Slide 3 — The Stakes 2014-2016 Blob: $100M+ fishery losses. 2019 Morro Bay OsHV-1 outbreak: documented oyster mortality at [specific number]. Frequency tripled since 1980s. This problem is getting worse.

Slide 4 — What Exists Today NOAA Blobtracker screenshot. Caption: "Tells you the heatwave is here. Doesn't tell you it's coming." Contrast with Tideline's 5-7 day forecast.

Slide 5 — Tideline, in Action Live dashboard screenshot pointed at San Diego Bay. Red region with "72-hour OsHV-1 risk: HIGH." This is the demo moment.

Slide 6 — How It Works (30 seconds) One-line architecture: Buoys + CalCOFI + Satellite → LightGBM/CNN ensemble → 10km grid forecast. Backtested AUC 0.70-0.80 on 2015 Blob. One sentence: "We're the first to fuse these sources at this granularity for this use case."

Slide 7 — Business Model

- Beachhead: California shellfish ($500-1500/mo per operator, 40 operators = $240k-720k ARR)
- Research partners: Scripps kelp labs (credibility, not revenue)
- Expansion: West Coast → National → Insurance data feed
- One line: "One prevented outbreak pays for 15 years of subscription."

Slide 8 — Why Us, Why Now

- Why us: UC Santa Cruz team, Scripps data partnership pathway, built it in 15 hours (proof of velocity)
- Why now: 2023 was warmest ocean year recorded, MHW frequency accelerating, aquaculture operators actively looking for forecasting tools
- Ask: introductions to 3 aquaculture operators for paid pilots

---

The 90-Second Version (for the actual demo)

> "Marine heatwaves are killing oysters. When ocean temps sustain 21°C, a virus called OsHV-1 activates and can wipe out a farm's juvenile stock in 48 hours. There's no treatment. The only defense is early harvest — but that requires 3 days of warning.
>
> Today, oyster farmers like Carlsbad Aquafarm rely on NOAA's buoy data and the Blobtracker dashboard. Both of these tell you a heatwave is happening now. By then, it's too late.
>
> Tideline fuses buoy, CalCOFI, and satellite data into a forecast model that predicts marine heatwave probability 5 to 7 days ahead, at 10 kilometer granularity. *[Switch to dashboard. Show 2015 Blob backtest.]* This is our model, trained only on data up to 2014, forecasting the 2015 Blob three days before it hit. Our backtest AUC is 0.70 to 0.80 on the held-out event.
>
> The business is vertical SaaS for aquaculture. Forty commercial shellfish operators on the California coast, $500 to $1500 per month. One prevented outbreak pays for 15 years of subscription. Year 2 expands to West Coast and an insurance data product — because parametric climate insurance is a $20 billion market that doesn't have an MHW forecast input yet.
>
> We built this in 15 hours on real Scripps data. We want introductions to aquaculture operators. Thanks."

Time that. 90 seconds, maybe 100. That's the pitch.

The One Thing to Do Tonight That Will 10x the Pitch

Email or call one aquaculture operator before you sleep.

Draft message you could send to Carlsbad Aquafarm, Hog Island Oyster Co, or Pacific Shellfish Growers Association right now:

> Hi [name],
>
> I'm a CS student at UC Santa Cruz. My team is building a marine heatwave forecasting tool aimed at helping shellfish operators get 5-7 day warning on thermal events that trigger OsHV-1 and other mortality risks. Before we go any further, I'd love 10 minutes of your time to ask whether early warning is even the right problem to solve, or if the real pain is somewhere else.
>
> Any time this week works. Thanks for considering it.
>
> [JD]

Even if you don't get a reply in 15 hours, **the email is sent**. In the pitch you say: "We've reached out to [name] at Carlsbad Aquafarm and the Pacific Shellfish Growers Association to validate the problem. That's our Monday morning." Judges treat "we emailed the customer" as 10x more credible than "we read a paper."

What to Cut From the Current Framing

- Drop "reef conservation" from the core pitch. It dilutes the wedge. Kelp labs are partners, not customers.
- Drop "broad NOAA tracking misses local granularity" as framing. It sounds defensive. Lead with the OsHV-1 thermal trigger instead — it's a better hook.
- Drop the generic "$100K+ per outbreak" until you can cite the specific event it's anchored to.
- Drop "auditable Marimo notebook and API enable quick integration" from the business pitch. That's technical-track language. For the business track, translate it to: "We ship with a customer-facing API so operators can pipe forecasts directly into their own farm management systems."

What the VC Panel Would Say After This Revision

- **Climate Tech Generalist:** "Okay, this is a real wedge. The OsHV-1 trigger makes it operational, not academic. I'd take the meeting."
- **Vertical SaaS Investor:** "Forty customers in Year 1 is small but the expansion story is clean. Prove you can close 3."
- **Enterprise/Insurance Investor:** "Insurance is the prize, and I like that you named it as Year 2 instead of Year 1. Keep me posted."
- **Data Infrastructure Investor:** "Fused regional MHW forecasts becoming the canonical input for downstream climate-adaptation software — I can see the picks-and-shovels version. Build the customer base first, infrastructure pitch later."
- **Skeptic:** "Did you talk to the operator?" *[You say yes, you emailed them.]* "Fine. Show me the dashboard."

a clear idea on the project

In Marine Context, marine heatwaves—prolonged SST spikes above the **90th** percentile for 5+ days—are intensifying in frequency and severity off Southern California due to climate change. The 2013-2016 Pacific Blob exemplified this, causing **$3-4B** in fishery collapses, seabird die-offs, and toxic algal blooms by disrupting ocean ecosystems. San Diego's aquaculture and kelp restoration now face acute risks, like OsHV-1 oyster viruses thriving in 21-24°C waters, without adequate lead-time warnings.

Current Solutions like NOAA's Blobtracker provide real-time detection of marine heatwaves via SST anomalies but **lack** 5-7 day lead-time forecasts needed for San Diego operators to act. Their coarse resolution (1/4° grids) misses local Southern California Bight dynamics, while physics-based models update slowly due to public mandates and no multi-source data fusion (buoy + CalCOFI + satellite). Reactive monitoring post-event fails to prevent losses like oyster die-offs, as no tool delivers granular, probabilistic predictions for aquaculture decisions.

What Tideline Solves the climate-driven marine heatwave crisis by delivering actionable 5-7 day probabilistic forecasts at 10km coastal granularity for the Southern California Bight. It fuses NOAA buoys (hourly point data), Scripps CalCOFI (quarterly profiles), and satellite SST rasters into unified features—overcoming single-source limitations—to predict MHW onset before SST crosses the 90th percentile threshold.
This enables San Diego aquaculture operators to harvest oysters pre-OsHV-1 outbreak and kelp restorers to deploy mitigations, directly preventing $100K+ losses per event that current detection-only tools miss.

NOAA/Scripps haven't done it because their mandate prioritizes broad scientific research and global detection (e.g., Blobtracker maps current anomalies) over commercial-grade, hyper-local 5-7 day forecasts for specific users like San Diego farms—physics models are slow, experimental subseasonal outlooks lack coastal precision, and public funding favors open data over proprietary fusion.
Tideline can by agilely fusing disparate public sources (buoys, CalCOFI, satellites) with ML ensembles (LightGBM + CNN) in Databricks, iterating weekly on a narrow Bight grid via hackathon-proven pipeline—delivering auditable API/dashboard faster than bureaucratic research cycles.

Tideline evolves from hackathon feature to organization by monetizing proprietary fusion of public data into customer-specific forecasts, building a recurring revenue SaaS for underserved San Diego operators ignored by NOAA's broad research.
It leverages agile ML iterations (Delta Lake versioning, weekly retrains on user feedback) and data moat (Bight-specific features + private sensors), scaling via API to CA-wide fisheries while public tools stay detection-only.

Key San Diego Operators

- San Diego Bay Aquaculture (Port of San Diego Blue Economy Incubator): Grows juvenile oysters in FLUPSY systems in nutrient-rich bay waters; Tideline's 5-7 day MHW forecasts prevent OsHV-1 viral outbreaks during heat spikes (21-24°C), enabling early harvest to avoid mass die-offs.
- **Ola Farms (Grape St. Pier, San Diego Bay): Regeneratively farms seaweed and shellfish for ocean bioremediation; forecasts allow preemptive protection of crops from SST anomalies that degrade growth, cutting losses in restoration efforts.**
    - **Quantity/economic impact without the solution**
- SeaTrees (La Jolla kelp restoration with Scripps): Outplants kelp in San Diego County underwater forests; 5-7 day warnings mitigate heatwave-induced die-offs (e.g., post-Blob style), optimizing urchin removal and seeding timing.
- Sunken Seaweed/HyperKelp Inc. (Port incubator pilots): Cultivates native seaweed for carbon sequestration in bay; granular predictions safeguard pilot yields against warming events disrupting productivity.

Why They Benefit These operators lose revenue from sudden MHWs triggering viruses, kelp stress, or halted growth—current NOAA tools detect too late. Tideline's fused probabilities give lead time to move gear, harvest, or intervene, saving $100K+ per event in a $3B+ impact region

Problem statement
Market section
Solution
References for everything / statistics for farms and heatwaves

brev

https://jupyter0-2p320s3rg.brevlab.com

What to do:

1. Have teammates open the shared Brev notebook/workspace.
2. In that environment, open a terminal (or run shell commands from a notebook cell).
3. Clone/open the repo and run the project scripts directly.
4. Commit code changes to git, not notebook cells.

Recommended teammate workflow:

1. Open project root and run setup
    - python -m venv .venv
    - source .venv/bin/activate
    - pip install -e .
2. Set env vars in that Brev session
    - GCS_BUCKET
    - optional MLflow/Databricks vars
3. Run the training script you already have:
    - brev_train_raster_cnn.sh
4. Save outputs to shared storage (GCS) and push code changes to git.